

Computer Programming

```

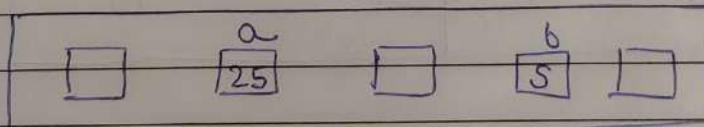
1. # include <stdio.h>
2.
3. int main ( ) {
4.     printf ("Hello world");
5.     return 0;
6. }
```

CH = 1

- | | |
|--------------|----------------|
| 1) Variable | 5) Structure |
| 2) Keywords | 6) Compilation |
| 3) Constants | |
| 4) Comments | |

★ variables :- variable is the name of memory location which stores some data.

Memory :-



If we store any numbers, char star, age

```
#include <stdio.h>
```

```
int main ( ) {
```

```
    int number = 25 ;
```

```
    int char star = '*' ;
```

```
    int age = 22 ;
```

```
    return 0 ;
```

```
}
```

★ Rules of variable

- a) Variables are case sensitive
for eg :- `int a = 30;` `int A = 40;` Both are different
- b) 1st character is alphabet or '-'
- c) No comma / blank space
- d) No other symbol other than '-'

★ Data types

Data types	Size in bytes
char or signed char	1
Unsigned char	1
int or signed int	2
unsigned int	2
Short int	2
Long int	4
float	4
Double	8
Long double	10

```
#include <stdio.h>
```

```
int main () {
```

```
int age = 22;
```

```
float pi = 3.14;
```

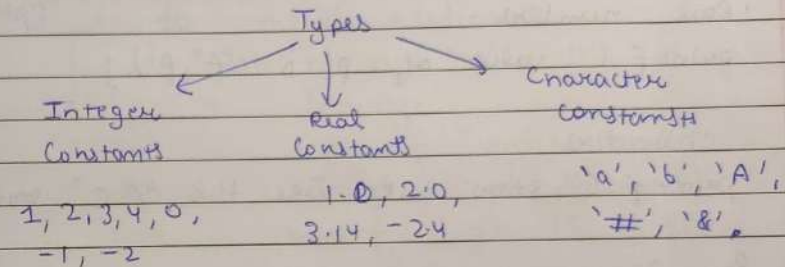
```
char nstag = '#';
```

```
return 0;
```

```
}
```

★ Constants :-

Values that don't change (fixed)



★ Keywords

Reserved words that have special meaning to the compiler. There are 32 keywords in C.

★ Comments

Lines that are not part of program

single line

```
//
```

multiple line

```
/*
*/
```

★ Output

```
printf("Hello world");
```

new line

```
printf("Kuch bhi\n");
```


cases of output

1. Integers

printf("age is %d, age);

2. Real numbers

printf("value of pi is %f", pi);

3. characters

printf("stack looks like this %c", stack);

Low eg :-

#include <stdio.h>

int main () {

int age = 22;

printf("age is %d", age);

return 0;

}

~~in~~

#include <stdio.h>

int main () {

float pi = 3.14;

printf("age is %f\n", pi);

return 0;

}

Format
Specifier

Input

scanf("%d", &age);

↳ int

#include <stdio.h>

int main () {

int age;

printf("enter age");

scanf("%d", &age);

printf("age is : %d", age);

return 0;

}

#include <stdio.h>

int main () {

int a, b;

printf("enter a");

scanf("%d", &a);

printf("enter b");

scanf("%d", &b);

~~int main () {~~

printf("sum is : %d", a+b);

return 0;

}

Q. Compilation :-

A computer program that translates C code into machine code.

hello.c $\rightarrow$ C Compiler $\rightarrow$ a.exe (windows)
a.out (linux & mac)

Practice ques

1. Write a program to calculate area of a square (side is given).

```
#include <stdio.h>
int main() {
    int side;
    printf("enter side");
    scanf("%d", &side);

    printf("area is : %d", side * side);
    return 0;
}
```

2. Write a program to calculate area of a circle (radius is given).

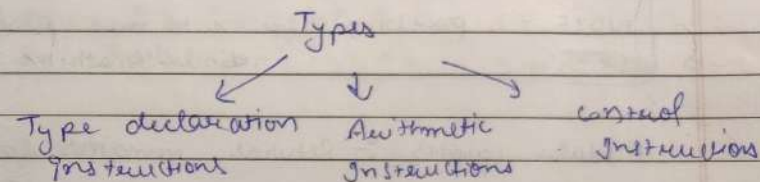
```
#include <stdio.h>
int main() {
    float radius;
    printf("enter radius");
    scanf("%f", &radius);
}
```

```
printf("area is : %.f", 3.14 * radius * radius);
return 0;
}
```

Ch :- 2

Instructions & operators

Instructions :-



Type declaration $\rightarrow$ Declare ~~before~~ variable before using it.

valid

```
int a = 4; ✓
int b = a;
int c = b + 1;
int d = 1, e;
```

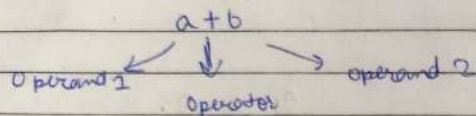
```
int a, b, c;
a = b = c = 1;
```

Invalid

```
int a = 4;
int b = a;
int b + 1 = c;
int d = 2, e;
```

```
int a, b, c = 1;
```


Arithmetic Instructions



NOTE - single variable on the LHS

NOTE - $\text{pow}(x, y)$ for x to the power y
`<include math.h>`

Modulus operator :- Returns remains for int

$$3 \% 2 = 1$$

$$-3 \% 2 = -1$$

$$3 \% -2 = 1$$

Type conversion

int / $\rightarrow$ int

float / $\rightarrow$ float

int op int $\rightarrow$ int

float op float $\rightarrow$ float

int op float $\rightarrow$ float

Operator precedence

*, /, %

$\downarrow$

+, -

$\downarrow$

=

$$x = 4 + 9 * 10$$

$$x = 4 + 90 = 94$$

$$x = 4 * 3 / 6 * 2$$

$$x = 12 / 12 = 1$$

Associativity (for same precedence)

$\rightarrow$ left to right

$$x = 4 * 3 / 6 * 2$$

$$= 2 * 2 = 4$$

Practice ques

a) $5 * 2 - 2 * 3 = 10 - 6 = 4$

b) $5^{**}2 / 2 * 3 = 15$

c) $5 * (2 / 2) * 3 = 5 * 1 * 3 = 15$

d) $5 + 2 / 2 * 3 = 5 + (2 / 2) * 3 = 5 + (1 * 3) = 5 + 3 = 8$

In exercise

ques :- [F]

a) Sol :-

$$K = 2 / 3 * 3$$

$$= 0$$

$$J = 3 / 2 * 2$$

$$= 2$$

$$a = 2 / 3 * 3$$

$$= 0.00$$

$$b = 3 / 2 * 2$$

$$= 2.000$$

final output

~~0 2000~~

0 2 0.000 2.0000

$$3 * 2.21$$

$$253$$

$$47.5$$

$$5 \sqrt{4}$$

Sol: - b)

$$a = -3 + 3$$

$$= 0$$

$$b = -3 - 3$$

$$= -6$$

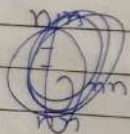
Final output
0 -6

c) C = 1

Final output
1

e) Enter value of a and b

f) Output



nn

nn
nn/n/n nn/n

* Control Instructions :-

Used to determine flow of current

- a) Sequence control
- b) Decision control → if, else
- c) Loop control → for loop, while loop
- d) Case control

* Operator

- a) Arithmetic operators → +, -, /, *, %
- b) Relational operators
- c) Logical "
- d) Bitwise "
- e) Assignment "
- f) Ternary "

* Relational operators

= (equal to) a = b 0 True → 1
False → 0

>, >= 3 > 4 → 0, 4 > 3 → 1
4 > 4 → 0, 4 ≥ 4 → 1

<, <= 3 < 4 → 1, 3 < 3 → 0
4 < 3 → 0, 2 < 3 → 1

!= (not equal to) True → 1
False → 0

ALL AND

11 or

1 NOT

T F

$(A > 3) \wedge (2 > 5)$

↓ ↓

f output 150

1

2

Out	1	2
T	T	T
F	F	T
F	T	F
F	F	F

|| (OR) $\rightarrow$ It is opposite of & AND

out	1	2
T	T	T
T	T	F
T	F	T
F	F	F

↑ NOT →

1 (3 > 4)

2

(57)

F

Q

for eg

```
printf("%d\n", !((5>1) && (3>4)));
```

output $\rightarrow 1$

Assignment Operator

2

$$a = 6$$

Δ
C20

$$c = f + g + h$$
 $+$
$$a = a + b$$
$$\rightarrow a + b$$

Sum as last

- =

* =

 $L =$
$$0/0 =$$

Q

Write a program check if a number is divisible by 2 or not.

let number $x \pmod{2} \rightarrow 0 \text{ (Div)}$

(imp)

(imp) $\rightarrow$ $\begin{array}{l} 1 \\ 2 \\ -1 \\ -1 \end{array}$ (not divide)

```
# include <stdio.h>
```

```
int main ( ) {
```

```
int x;
```

```
printf ("enter a number : ");
```

```
scanf ("%d", &x);
```

```
printf ("v: %d, n %d == 0");
```

~~5 return 0;~~

- a) write a program to check if the no is even or odd.

```
#include <stdio.h>
```

```
int main () {
```

```
// even → 1
```

```
// odd → 0
```

```
int x;
```

```
printf ("enter a number");
```

```
scanf ("%d", &x);
```

```
printf ("%d", x % 2 == 0);
```

```
return 0;
```

```
}
```

Practice

- a) if it's Sunday & its snowing → (true)

```
int main () {
```

```
int isSunday = 1;
```

```
int isSnowing = 1;
```

```
printf ("%d", isSunday && isSnowing);
```

```
return 0;
```

```
}
```

Output : 1

- b) If its Monday or its raining → (true)

```
int main () {
```

```
int isMonday = 0;
```

```
int isRaining = 1;
```

```
printf ("%d\n", isMonday || isRaining);
```

```
return 0;
```

```
}
```

(Output → 1)

- c) If a number is greater than 9 & less than 100 → true (2 digit number)

```
int main () {
```

```
int x;
```

```
printf ("enter a number");
```

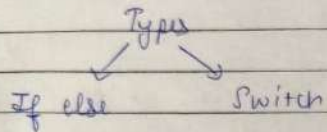
```
scanf ("%d", &x);
```

```
printf ("%d\n", x > 9 && x < 100);
```

```
return 0;
```

```
}
```


* Conditional Statement :-



* if-else

```

if (condition) {
    // do something if TRUE
}
else {
    // do something if FALSE
}
  
```

For eg :-

```

int main
#include <stdio.h>
int main () {
    int age;
    printf ("enter age");
    scanf ("%d", &age);
    if (age > 18) {
        printf ("adult\n");
    }
    else {
        printf ("not adult\n");
    }
    return 0;
}
  
```

else if

```

if (condition 1) {
    // do something if TRUE
}
else if (condition 2) {
    // do something if 1st is false & 2nd is TRUE.
}
  
```

For eg :-

```

int #include <stdio.h>
int main () {
    int age;
    printf ("enter age");
    scanf ("%d", &age);
    if (age >= 18) {
        printf ("adult\n");
    }
    else if (age > 13 && age < 18) {
        printf ("teenager\n");
    }
    else {
        printf ("child\n");
    }
    return 0;
}
  
```


Conditional operators :-

* Ternary

Condition ? do something if TRUE : do something if FALSE
 ↓
 check

For eg :-

```
# include <stdio.h>
int main () {
    int age;
    printf ("enter age");
    scanf ("%d", &age);
    age >= 18 ? printf ("adult\n") : printf ("not adult\n");

    return 0;
}
```

* Switch

```
Switch (number) {
Case 1: // do something
    break;
Case 2: // do something
    break;
default: // do something
}
```

case 'a' :	case 1 :
case 'b' :	case 2 :
case 'c' :	case 3 :

For eg :-

```
# include <stdio.h>
int main () {
    int day; // 1-mon; 2-tue; 3-wed
    printf ("enter day (1-7)");
    scanf ("%d", &day);

    Switch (day) {
case 1: printf ("monday\n");
        break;
case 2: printf ("Tuesday\n");
        break;
case 3: printf ("Wednesday\n");
        break;
case 4: printf ("Thursday\n");
        break;
case 5: printf ("Friday\n");
        break;
case 6: printf ("Saturday\n");
        break;
case 7: printf ("Sunday\n");
        break;
default: printf ("not valid day!\n");
    }

    return 0;
}
```


Switch operators properties

- Cases can be in any order.
- Nested Switch (Switch inside switch) are allowed.

★ Nested if :-

```
#include <stdio.h>
int main;
int number;
printf("enter number");
scanf("%d", &number);
if (number >= 0) {
    printf("positive\n");
    if (number % 2 == 0) {
        printf("even\n");
    } else {
        printf("odd\n");
    }
} else {
    printf("negative\n");
}
```

Ques :- write a program to check if a student passed or failed.
 marks > 30 is PASS
 marks <= 30 is FAIL

- Q) write a program to give grades to a student marks < 30 is C.

30 <= marks < 70 is B.

70 <= marks < 90 is A.

90 <= marks <= 100 is A++.

```
#include <stdio.h>
```

```
int main () {
```

```
    int marks;
```

```
    printf("enter marks (0-100) : ");
```

```
    scanf("%d", &marks);
```

```
    if (marks < 30) {
```

```
        printf("C\n");
```

```
    }
```

```
    else if (marks >= 70 && marks < 90) {
```

```
        printf("A\n");
```

```
    }
```

```
    else if (marks >= 30 && marks < 70) {
```

```
        printf("B\n");
```

```
    }
```

```
    else {
```

```
        printf("A+\n");
```

```
    }
```

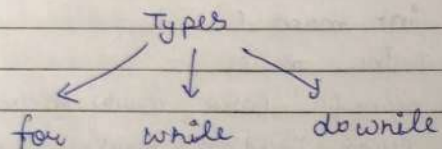
```
    return 0;
```

```
}
```


Q) write a program to find if a character entered by user is upper case or not.

* loop control Instructions :-

To repeat some parts of the program.



For loop

for (initialisation ; condition ; updation) {

// do something

}

for eg :-

```

for (int i=1 ; i<=5 ; i=i+1) {
    printf("HW");
}
  
```

Syntax :-

```

#include <stdio.h>
int main () {
    for (int i=1 ; i<=5 ; i=i+1) {
        printf("helloworld");
    }
    return 0;
}
  
```

If we want to print numbers 1 to 100 then code is

```

#include <stdio.h>
int main () {
    for (int i=1 ; i<=100 ; i=i+1) {
        printf("%d\n", i);
    }
    return 0;
}
  
```

Q) Print the Number 0 to 10 .

```

#include <stdio.h>
int main () {
    for (int i=0 ; i<=10 ; i=i+1) {
        printf("%d\n", i);
    }
    return 0;
}
  
```

There are two types of

i) Increment operator ~~(i++)~~ & ~~(++i)~~
(i++) use, then increase (Post increment)

```

int i = 1;
printf("%d\n", i++);
printf("%d\n", i);
Output :- 1
          2
  
```


ii) Increment operator

(++i) increase then use

int i = 1

printf("%d\n", ++i);

printf("%d\n", i);

output → 2

2

★ Decrement operator

i) (--i) pre-decrement

ii) (i--) post decrement

int i = 1

printf("%d\n", i--);

printf("%d\n", i);

output → 1

0

int i = 1

printf("%d\n", --i);

printf("%d\n", i);

output → 0

0

Special Things

- Increment Operator
- Decrement Operator
- loop counter can be float or even character.
- Infinite loop.

Infinite loop

int main () {

for (int i = 1; ; i++) {

printf("hello world\n");

}

return 0;

}

loop counter can be float or even character

int main () {

for (float i = 1.0; i <= 5.0; i++) {

printf("%f\n", i);

}

for (char ch = 'a'; ch <= 'z'; ch++) {

printf("%c\n", ch);

}

★ while loop

while (condition) {

// do something

}

for eg :- #include <stdio.h>

int main () {

int i = 1;

while (i <= 5) {

printf("%d\n", i);

i++

return 0;

Q) Print the numbers from 0 to n, if n is given by user $n=4$.

```
#include <stdio.h>
int main () {
    int n;
    printf ("enter number : ");
    scanf ("%d", &n);

    int i = 0;
    while (i <= n) {
        printf ("%d\n", i);
        i++;
    }

    return 0;
}
```

* do while loop :-
do {
// do something
} while (condition);

For eg :-

```
#include <stdio.h>
int main () {
    int i = 1;
    do {
        printf ("%d\n", i);
        i++;
    } while (i <= 5);
}
```

For eg :-

```
#include <stdio.h>
int main () {
    int i = 1;
    do {
        printf ("%d\n", i);
        i++;
    } while (i <= 5);

    return 0;
}
```

Q) Print the sum of first n Natural Numbers.
 $n=4$.

```
#include <stdio.h>
int main () {
    int n;
    printf ("enter number n");
    scanf ("%d", &n);

    for (int i = 1; i <= n; i++) {
        printf ("%d\n", i);
    }

    return 0;
}
```



```
#include <stdio.h>
int main () {
    int n;
    printf ("enter number : ") ;
    scanf ("%d", &n);
```

```
    int sum = 0;
    for (int i=1; i<=n; i++) {
        sum = sum + i;
    }
    printf ("sum is %d", sum i);
    return 0;
```

}

OR

```
int n;
printf ("enter value of n");
scanf ("%d", &n);
int sum = 0;
for (int j = n; j >= 1, j--) {
    sum = sum + j;
}
printf ("sum is %d", sum);
return 0;
```

}

~~Sum~~

Q) Print the table on the program

```
#include <stdio.h>
int main () {
    int n;
    printf ("Enter number n");
    scanf ("%d", &n);
```

```
    for (int i=1; i<=10; i++) {
        printf ("%d \n", n*i);
    }
    return 0;
```

}

★ ~~break~~ break statement
↓
exit the loop

For eg :-

```
#include <stdio.h>
int main () {
    for (int i=1; i<=5; i++) {
        if (i==3) {
            break;
        }
        printf ("%d", i);
    }
    printf ("end");
    return 0;
```

}

Output is
1
2
end

- Q2) keep taking numbers as input from user until user enters an odd number.

```
#include <stdio.h>
int main()
{
    int n;
    do {
        printf("Enter number");
        scanf("%d", &n);
        printf("%d\n", n);
        if (n % 2 != 0) {
            break;
        }
        while (1);
        printf("No. is odd");
    }
    return 0;
}
```

break

Continue statement

↓
Skip to next iteration

for eg:-

```
#include <stdio.h>
int main()
{
    for (int i = 1; i <= 5; i++) {
        if (i == 3)
            continue;
    }
}
```

```
printf("%d\n", i);
}
```

```
return 0;
}
```

Output

1

2

4

5

- Q3) Print all no from 1 to 10 except 6.

```
#include <stdio.h>
int main()
{
    for (int i = 1; i <= 10; i++) {
        if (i == 6)
            continue;
    }
    printf("%d", i);
}
return 0;
}
```

- Q4) Print out the odd no. from 1 to 50.

```
#include <stdio.h>
int main()
{
    for (int i = 1; i <= 50; i++) {
        if (i % 2 != 0)
            printf("%d\n", i);
    }
    return 0;
}
```


Q) Print the factorial of a no. n.

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter number ");
    scanf("%d", &n);
    int fact = 1;
    for (int i = 1; i <= n; i++)
        fact = fact * i;
    printf("Factorial is %d", fact);
    return 0;
}
```

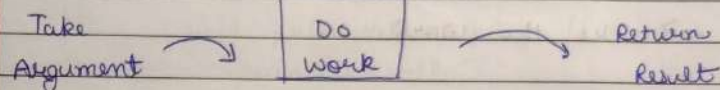
Q) calculate the sum of the no. b/w 5 and 50 (include 5 and 50).

```
#include <stdio.h>
int main()
{
    int sum = 0;
    for (int i = 5; i <= 50; i++)
        sum = sum + i;
    printf("sum is %d", sum);
    return 0;
}
```

* Function & Recursion :-

Functions

block of code that performs particular task.



- It can be used multiple times.
- Increase code reusability.

Syntax 1

Function prototype

void print Hello(); ←

↓
No any return value

> Tell the compiler

Syntax 2

Function Definition

void print Hello();

↓
printf("Hello"); ←

↓

> Do the work

Syntax 3 Function call

```
int main () {
    printHello(); ←
    return 0;
}
```

> Use the work

for eg :-

```
#include <stdio.h>
void printHello();

int main () {
    printHello(); // Function Definition
    printf("Hello");
    printf("Hello");
    printf("Hello");
    return 0;
}

void printHello () {
    printf
```

for eg :-

```
#include <stdio.h>
void printHello();
```

```
int main () {
    printHello(); // Function call
    print return 0;
}

void printHello () { // Function Definition
    printf("Hello");
}
```

Q) write two functions - one to print "Hello" & second to ~~give~~ print "good bye".

```
#include <stdio.h>
void printHello(); // Function prototype
void printGoodbye();
```

```
int main () {
    printHello(); // Function call
    printGoodbye();
    return 0;
}

void printHello () { // Function Definition
    printf("Hello");
    printf("good bye\n");
}

void printGoodbye () {
    printf("good bye\n");
}
```


Q) write a function that prints Namaste if user is Indian & Bonjour if the user is french.

```
#include <stdio.h>
void namaste ();
void bonjour ();

int main ()
{
    char ch;
    printf ("enter f for french and i for indian");
    scanf ("%c", &ch);

    if (ch == 'i') {
        namaste ();
    }
    else {
        bonjour ();
    }

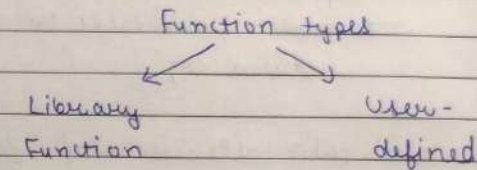
    return 0;
}

void namaste () {
    printf ("Namaste\n");
}

void bonjour () {
    printf ("Bonjour\n");
}
```

★ Properties of function :-

- i) Execution always starts from main.
- ii) A function gets called directly or indirectly from main.
- iii) There can be multiple functions in a program.



Special functions
inbuilt in C

declared & defined
by programmer

scanf(), printf()

★ Passing Argument

Functions can take value & give some value.

```

graph TD
    A[Functions can take value & give some value] --> B[Parameter]
    A --> C[return value]
  
```

- i) void print hello (); ←
- ii) void print Table (int n); ←
- iii) int sum (int a, int b); ←


```
#include <stdio.h>
void print table (int n);

int main () {
    int n;
    printf("Enter number n:");
    scanf("%d", &n);
    return; print table (n); // argument
    return 0;
}
```

```
void print table (int n) { // parameter
    for (int i = 1, i <= 10, i++)
        printf("i * n"); (" %d\n", i * n);
}
```

☆ Recursion

When a function call itself, is called Recursion.

For eg :- Print "hello world" 5 times.

```
#include <stdio.h>
```

```
void print Hw (int count);
int main () {
    print Hw (5);
    return 0;
}
```

// recursive function //

```
void print Hw (int count) {
    if (count == 0);
    return;
}
```

```
void print printf ("hello world\n");
    print Hw (count-1);
```


Maths

$$f(n) = n^2$$

$$f(1) = 1^2 = 1$$

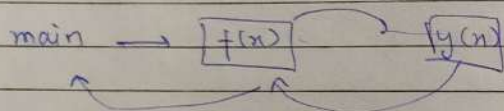
$$f(2) = 2^2 = 4$$

$$f(3) = 3^2 = 9$$

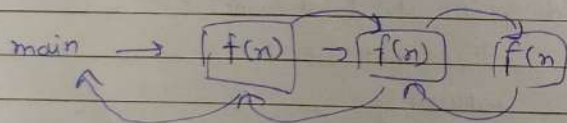
$$f(f(n)) = f(2^2) = 4^2 = 16$$

$$f(f(f(n))) = f(f(4)) = f(16) = 256$$

Normal function call



Recursion function call



Q) Sum of first n natural numbers

$$\begin{aligned}
 n &\rightarrow 1 + 2 + 3 + 4 + \dots + n-1 + n = \text{sum}(n-1) + n \\
 n=5 &\rightarrow 1 + 2 + 3 + 4 + 5 = \text{sum}(4) + 5 \\
 n=4 &\rightarrow 1 + 2 + 3 + 4 = \text{sum}(3) + 4 \\
 n=3 &\rightarrow 1 + 2 + 3 = \text{sum}(2) + 3 = \text{sum}(1) + 3 \\
 n=2 &\rightarrow 1 + 2 = \text{sum}(1) + 2 \\
 n=1 &\rightarrow 1
 \end{aligned}$$

```

#include <stdio.h>
int sum(int n);

int main() {
    printf

```

```

// recursive function
int sum1 = sum(n-1);
int N = sum(n-1) + n

```

```

#include <stdio.h>
int sum(int n);

```

```

int main() {
    printf("sum is : %d", sum(5));
    return 0;
}

```

```

// recursive function
int sum(int n) {
    if (n == 1) {
        return 1;
    }
    int sumN1 = sum(n-1);
    int sumN = sum(n-1) + n;
    return sumN;
}

```


$$\begin{aligned}
 \text{sum}(5) &= 15 \\
 \swarrow \quad \searrow \\
 \text{sum}(4) + 5 &= 10 + 5 = 15 \\
 \swarrow \quad \searrow \\
 \text{sum}(3) + 4 &= 6 + 4 = 10 \\
 \swarrow \quad \searrow \\
 \text{sum}(2) + 3 &= 3 + 3 = 6 \\
 \swarrow \quad \searrow \\
 \boxed{\text{sum}(1) + 2} &= 3 \\
 \swarrow \quad \searrow &\rightarrow \text{base case} \\
 1 &
 \end{aligned}$$

② Factorial of n

$$\begin{aligned}
 5! &= 1 \times 2 \times 3 \times 4 \times 5 = 4! \times 5 \\
 4! &= 1 \times 2 \times 3 \times 4 = 3! \times 4 \\
 3! &= 1 \times 2 \times 3 = 2! \times 3 \\
 2! &= 1 \times 2 = 1! \times 2 \\
 1! &= 1 = 1 \\
 0! &= 1
 \end{aligned}$$

$$\text{fact}(n) = \text{fact}(n-1) \times n$$

```

#include <stdio.h>
int fact (int n);

int main () {
    printf ("Fact of N : %d", fact(5));
    return 0;
}

```

```

// recursive function
int fact (int n) {
    if (n == 1) return 1;
    int fact N = fact (n-1);
    return fact N * n;
}

```

$$\begin{aligned}
 \text{fact}(5) &= 120 \\
 \swarrow \quad \searrow \\
 \text{fact}(4) \times 5 &= 120 \\
 \swarrow \quad \searrow \\
 \text{fact}(3) \times 4 &= 24 \\
 \swarrow \quad \searrow \\
 \text{fact}(2) \times 3 &= 6 \\
 \swarrow \quad \searrow \\
 \text{fact}(1) \times 2 &= 2 \\
 \swarrow \quad \searrow \\
 1 &= 1
 \end{aligned}$$

Uses of Recursion :-

Anything that can be done with iteration, can be done with recursion and vice-versa.

- Recursion can sometimes give the most simple solution.
- Base case is the condition which stops recursion.
- Iteration has infinite loop & Recursion has stack overflow.
- Write a function to print n terms of the fibonacci sequence.

fib(0) fib(1) fib(2) fib(3) fib(4) fib(5) fib(6) fib(7) fib(8)
0, 1, 1, 2, 3, 5, 8, 13, 21, ...

$$fib(2) = fib(0) + fib(1) \Rightarrow f(n) = fib(n-1) + fib(n-2)$$

$$fib(3) = fib(1) + fib(2)$$

$$fib(4) = fib(2) + fib(3)$$

```
#include <stdio.h>
```

```
int fib(int n);
```

```
int main () {
```

```
printf("Fibonacci of n: %d", fib(5));
```

```
return 0;
```

```
}
```

```
// recursive function //
```

```
int fib(int n) { if (n == 0); return 0; if (n == 1); return 1;
```

```
}
```

```
}
```

```
int fibNm1 = fib(n-1);
```

```
int fibNm2 = fib(n-2);
```

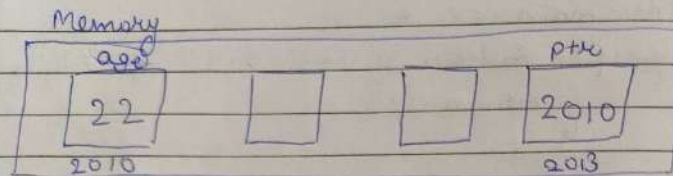
```
int fib(N) = fib(n-1) + fib(n-2);
```

```
return fib N;
```

```
}
```

- Write a function to find sum of digits of a number.

☆ Pointers :- A variable that stores the memory address of another variable.



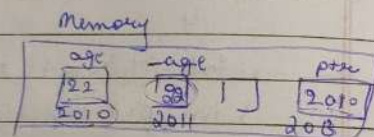
Syntax

```
int age = 22;
int *ptr = &age;
int age = *ptr;
```

* means value at address

For eg :-

```
#include <stdio.h>
int main () {
    int age = 22;
    int *ptr = &age;
    int age = *ptr;
    printf ("%d", age);
    return 0;
}
```



☆ Declaring Point

```
int *ptr;
char *ptr;
float *ptr;
```

```
int age = 22;
int *ptr = &age;
```

☆ Format Specifier

```
printf ("%p", &age);
printf ("%p", ptr);
printf ("%p", &ptr);
```

For eg :-

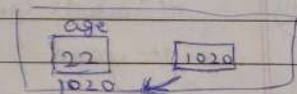
```
#include <stdio.h>
int main () {
    int age = 22;
    int *ptr = &age;
    // address
    printf ("%p", &age);
    printf ("%p", &age);
    return 0;
}
```

Format Specifier

```
printf ("%d", age); ← 22
```

```
printf ("%d", *ptr); ← 22
```

```
printf ("%d", *(&age)); ← 22
```



for eg :-

```
#include <stdio.h>
int main () {
    int age = 22;
    int *ptr = &age;

    // value
    printf("%d\n", age);

    printf("%d\n", *ptr);

    printf("%d\n", *(&age));

    return 0;
}
```

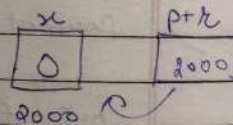
Q)

```
#include <stdio.h>
int main () {
    int x;
    int *ptr;
    ptr = &x;
    *ptr = 0;

    printf("x = %d\n", x); → 0
    printf("*ptr = %d\n", *ptr); → 0
}
```

```
*ptr += 5;
printf("x = %d\n", x); → 5
printf("*ptr = %d\n", *ptr); → 5
```

for eg :-



```
*ptr = *ptr + 5;
x = 5
x = 5
```

```
*(ptr)++;
printf("x = %d\n", x); → 6
printf("*ptr = %d\n", *ptr); → 6
```

$*ptr = *ptr + 1$
 $x = 5 + 1$
 $x = 6$

return 0;

Find output of this

Output is

$x = 0$

~~$*ptr = 0$~~

$x = 5$

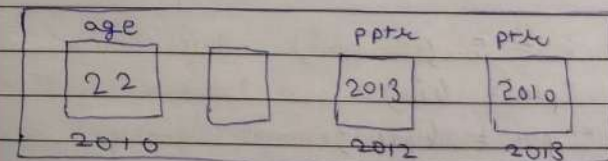
$*ptr = 5$

$x = 6$

$*ptr = 6$

★ **Pointer to Pointer** :- A variable that stores the memory address of another pointer.

memory



Pointer to Pointer

★ Syntax

```
int **pptr;
```


char ** pptr;

float ** pptr;

for eg :-

```
#include <stdio.h>
int main () {
    float price = 100.00;
    float * ptr = &price;
    float ** pptr = &ptr;

```

```
    return 0;
}
```

Q. Print the value of 'i' from its pointer to pointer.

```
#include <stdio.h>
int main () {

```

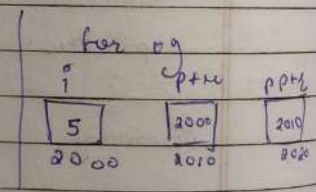
```
    int i = 5;
    int * ptr = &i;
    int ** pptr = &ptr;

```

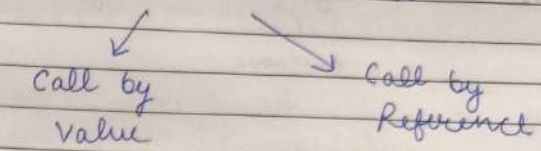
```
    printf("%d\n", **pptr);

```

```
    return 0;
}
```



★ Pointer in Function call



we pass value of variable as argument.

we pass address of variable as argument.

for eg :-

```
#include <stdio.h>
void square (int n);

```

```
int main () {
    int number = 4;
    square (number);
    printf("number = %d\n", number);
    return 0;
}

```

```
void square (int n) { // call by value
    n = n * n;
    printf("square = %d\n", n);
}

```

Output is

16
4

Q Arrays :- Collection of similar data types
stores at contiguous memory locations.
↓
continuous

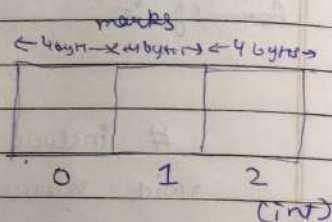
For eg :-

```
#include <stdio.h>
```

```
int marks[] = {97, 98, 89};
```

Syntax

```
int marks[3];
char name[10];
float price[2];
```



Input & Output

```
scanf("%d", &marks[0]);
```

```
printf("%d", marks[0]);
```

For eg :-

```
#include <stdio.h>
int main() {
    int marks[3];
    printf("Enter phy: ");
    scanf("%d", &marks[0]);
```

```
printf("Enter chem: ");
scanf("%d", &marks[1]);
```

```
printf("Enter math: ");
scanf("%d", &marks[2]);
```

```
printf("phy = %d, chem = %d, math = %d", marks[0], marks[1], marks[2]);
```

```
return 0;
}
```

Q Write a program to enter price of 3 items & print their final cost with gst.

```
#include <stdio.h>
```

```
int main() {
```

```
float price[3];
```

```
printf("Enter 3 prices: ");
```

```
scanf("%f", &price[0]);
```

```
scanf("%f", &price[1]);
```

```
scanf("%f", &price[2]);
```

```
printf("Total price 1: %f", price[0] + 0.18 * price[0]);
```

```
printf("Total price 2: %f", price[1] + 0.18 * price[1]);
```

```
printf("Total price 3: %f", price[2] + 0.18 * price[2]);
```

```
return 0;
```

```
}
```


Initialization of Array

```
int marks[3] = {97, 98, 89};
```

```
int marks[3] = {97, 98, 89};
marks: 4 → 4 → 4 →
```

97	98	89
0	1	2

int + 4

memory reserved : 12 bytes

for eg :-

```
char name[10]
```

↓

1 byte = $10 \times 1 = 10$ bytes

```
int roll[30]
```

↓

4 byte = $30 \times 4 = 120$ bytes

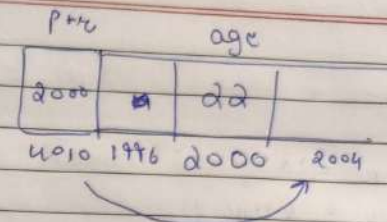
Pointer Arithmetic :-

Pointer can be incremented & decremented

CASE 1

```
int age = 22;
int *ptr = &age;
ptr++;
```

$ptr = ptr + 1 \rightarrow + \text{data type}$



ptr--;

For eg :-

```
#include <stdio.h>
```

```
int main() {
```

```
int age = 22;
```

```
int *ptr = &age;
```

```
printf("ptr = %u\n", ptr);
```

```
ptr++;
```

```
printf("ptr = %u\n", ptr);
```

```
ptr--;
```

```
printf("ptr = %u\n", ptr);
```

```
return 0;
```

```
}
```

Case 2 :-

```
float price = 20.00;
```

```
float *ptr = &price;
```

```
ptr++;
```


Case 3

```
char star = '*';
char *ptr = &star;
ptr++;
```

ptr++
one byte
because char
contains 1
byte

★ Pointer Arithmetic

i) we can also subtract one pointer from other.

ii) we can also compare two pointers.

for eg :-

#include <stdio.h>

int main () {

int age = 22;

int *ptr = &age;

int -age = 23;

int *-ptr = &-age;

printf (" difference = %u\n", ptr - -ptr);

-ptr = &-age

printf (" comparison = %u\n", ptr == -ptr);

return 0;

}

★ Array is a Pointer

int *ptr = &arr[0];

int *ptr = arr;

★ Array as Function argument

// Function Declaration

void printNumbers (int arr[], int n)

OR

void printNumbers (int *arr, int n)

// Function Call

printNumbers (arr, n);

for eg :-

#include <stdio.h>

void printNumbers (int arr[], int n);

int main () {

int arr[] = {1, 2, 3, 4, 5, 6};

printNumbers (arr, 6);

return 0;

}


```
void printNumbers (int arr [], int n) {
```

```
    for (int i=0; i<n; i++) {
        printf("%d\t", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

* Multidimensional Arrays

2D Arrays

```
int arr [][ ] = {21, 25, 23, 44}; // declare
```

```
// Access
```

```
0,0 0,1
```

```
1 2
```

```
arr [0][0] → 1
```

```
arr [0][1] → 2
```

```
arr [1][0] → 3
```

```
arr [1][1] → 4
```

```
3 4
1,0 1,1
```

For eg :-

```
# include <stdio.h>
```

```
int main ( ) {
```

```
    int marks [2][3];
```

```
    marks [0][0] = 90;
```

```
    marks [0][1] = 89;
```

```
    marks [0][2] = 98;
```

```
marks [1][0] = 78;
```

```
marks [1][1] = 99;
```

```
marks [1][2] = 100;
```

```
printf ("%d", marks [0][0]);
```

```
return 0;
```

```
}
```

Q

Write a function to count the no. of odd number in an array.

```
# include <stdio.h>
```

```
int countodd (int arr [], int n);
```

```
int main ( ) {
```

```
    int arr [] = {1, 2, 3, 4, 5, 6};
```

```
    printf ("%d", countodd (arr, 6));
```

```
    return 0;
```

```
}
```

```
int countodd (int arr [], int n) {
```

```
    for (int i=0; i<n; i++) {
```

```
        if (arr[i] % 2 != 0) {
```

```
            count++;
```

```
        }
```

```
    }
```

```
    return count;
```

```
}
```

} output :- 3

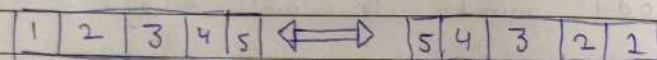
Q) `int arr[] = {1, 2, 3, 4, 5}`

for the given array, what will the following give?

a) `* (arr + 2)` → 3

b) `* (arr + 5)` → ~~error~~ 0

Q) write a function to reverse an array.
for eg:-



for `(int i=0, i < n/2, i++)`

first value = `arr[i]`

second value = `arr[n-1-i]`

0 `5-1-0 = 4`

1 `5-1-1 = 3`

2 `5-1-2 = 2`

Q) write a program to store the first n fibonacci numbers.

0, 1, 1, 2, 3, 5, 8, 13, 21,

#include <stdio.h>

int main () {

int n;

printf("Enter n:");

scanf("%d", &n);

int fib[n];

fib[0] = 0;

fib[1] = 1;

for (int i=2, i < n; i++) {

fib[i] = fib[i-1] + fib[i-2];

printf("%d\t", fib[i]);

}

return 0;

}

★ String :- A character array terminated by a "\0" (null character) null character denotes string terminator.

EXAMPLE :-

```
char name[] = {'S', 'H', 'R', 'A', 'D', 'H', 'A', '\0'};
```

```
char class[] = {'A', 'P', 'N', 'A', 'C', 'O', 'L', 'L', 'E', 'G', 'E', '\0'};
```

Initialising string :-

```
char name[] = {'A', 'N', 'K', 'U', 'S', 'H', '\0'};
```

```
char name[] = "ANKUSH";
```

```
char class[] = {'B', 'A', 'N', 'S', 'A', 'L', '\0'};
```

```
char class[] = "BANSAL";
```

Q What happens in memory?

```
char name[] = {'A', 'N', 'K', 'U', 'S', 'H', '\0'};
```

```
char name[] = "ANKUSH";
```

name

A	N	K	U	S	H	\0
2000	2001	2002	2003	2004	2005	2006

Q Create a string first Name & last Name to store details of user & print all the characters using a loop.

```
#include <stdio.h>
```

```
void printString (int arr[]);
```

```
int main () {
```

```
char firstName = "Ankush";
```

```
char lastName = "Bansal";
```

```
printString (firstName);
```

```
printString (lastName);
```

```
return 0;
```

```
}
```

```
void printString (char arr[]) {
```

```
for (int i = 0; arr[i] != '\0'; i++) {
```

```
printf ("%c", arr[i]);
```

```
}
```

```
printf ("\n");
```

```
}
```

★ String Format Specifiers

↓
"%s"

```
char name[] = "Ankush";
```

```
printf ("%s", name);
```


For eg :-

```
#include <stdio.h>
int main () {
    char name [50];
    scanf ("%s", name);
    printf ("your name is %s", name);
    return 0;
}
```

- Q) Ask the user to enter their first name & print it back to them.

```
#include <stdio.h>
int main () {
    char name [50];
    char firstName [50];
    scanf ("%s", firstName);
    printf ("your name is %s", firstName);
    return 0;
}
```

Also try this with their full name.

```
#include <stdio.h>
int main () {
    char full Name [100];
    fgets (full Name, 100, stdin);
    puts (full Name);
    return 0;
}
```

IMPORTANT

scanf() cannot input multi-word strings with spaces.

Here,
gets() & puts() comes into picture.

String Functions

gets(str) → Dangerous & outdated.

Input a string
(even multiword)

puts(str) =
output a string.

fgets(str, n, file)

Stops when n-1
chars input or new
line is entered.

★ Storing using Pointers

char *str = "Hello World";
store string in memory & the assigned address is stored in the char pointer 'str'.

char *str = "Hello World"; // can be reinitialized.

char str[] = "Hello World";
// cannot be reinitialized.

For eg :-

```
#include <stdio.h>
int main () {
    char *can change = "Hello World";
    puts (can change);
    can change = "Hello";
    puts (can change);
    return 0;
}
```

Output
Hello World
Hello

```
char cannot change = "Hello World";
puts (cannot change);
cannot change = "Hello";
↓
```

This gives error

Q) Make a program that inputs user's name & prints its length.

```
#include <stdio.h>
int countlength (char arr[]);

int main () {
    char name [100];
    gets (name, 100, stdin);
    printf ("length is : %d", countlength (name));
    return 0;
}

int countlength (char arr[]) {
    int count = 0;
    for (int i = 0; arr[i] != '\0'; i++) {
        count++;
    }
    return count - 1;
}
```

★ Standard Library Functions
↓
<string.h>

1. strlen (str)

count numbers of characters excluding '\0'.

for eg :-

```
#include <stdio.h>
#include <string.h>
int main () {
    char name[] = "Ankush";
    int length = strlen(name);
    printf("length is : %d", length);
    return 0;
}
```

output :- 6

2. strcpy (newstr, oldstr)

copies value of old string to new string.

for eg :-

```
#include <stdio.h>
#include <string.h>
int main () {
    char name[] = "Ankush";
    char lastname[] = "Bansal";
    strcpy(name, lastname);
    puts(name);
    return 0;
}
```

output :- Bansal

3. strcat (firststr, secstr) :-

Concatenates first string with second string.

for eg :-

```
#include <stdio.h>
#include <string.h>
int main () {
    char first name[100] = "Ankush";
    char lastname = "Bansal";
    strcat(name, lastname);
    puts(name);
    return 0;
}
```

output :- Ankush Bansal

4. strcmp (firststr, secstr) :-

Compares two strings & returns a value.

0 → string equal

positive → first > second (ASCII)

second →

negative → second > first (ASCII)

For eg :-

```
#include <stdio.h>
#include <string.h>
int main () {
    char firstName[] = "Ankush";
    char SecName[] = "Banjal";
    printf("%d", strcmp(firstName, SecName));
    return 0;
} output :- -1
```

Q) Salting

Find the salted form of a password entered by user if the salt is "123" & added at the end.

```
password = "test"
salt = "123"
newpass = "test123"
```

```
#include <stdio.h>
#include <string.h>
void salting (char password[];
```

```
int main () {
    char password[100];
    scanf("%s", password);
    salting (password);
}
```

```
void salting (char password[]) {
    char salt = "123";
    char newPassword[200];
    strcpy (newPassword, password);
    strcat (newPassword, salt);
    puts (newPassword);
}
```

Q) Write a function to count the occurrence of vowels in a string.

```
#include <stdio.h>
int countVowels (char str[]);
int main () {
    char str[] = "Ankush";
    printf ("Vowels are : %d", countVowels (str));
}
```

```
int countVowels (char str[]) {
    int count = 0;
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] == 'a' || str[i] == 'e' || str[i] == 'i'
            || str[i] == 'o' || str[i] == 'u') {
            count++;
        }
    }
    return count;
}
```


★ Structures :-

Structures is a collection of values of different data types.

Example :-

For a student store the following :-
name (String)
roll no (Integer)
cgpa (Float)

★ Syntax :-

```
struct student {  
    char name[100];  
    int roll;  
    float cgpa;  
};
```

```
struct student s1;  
s1.cgpa = 7.5;  
s1.name = ;  
s1.roll = ;
```

For eg :-

```
#include <stdio.h>  
#include <string.h>  
struct student {  
    int roll;  
    float cgpa;  
    char name[100];  
};  
int main () {  
    struct student s1;
```

```
s1.roll = 31;  
s1.cgpa = 9.5;  
strcpy(s1.name, "Ankur");
```

```
printf("Student Name = %s\n", s1.name);  
printf("Roll No. = %d\n", s1.roll);  
printf("cgpa = %f\n", s1.cgpa);  
  
return 0;  
}
```

★ Structures in memory

```
struct student {  
    char name[100];  
    int roll;  
    float cgpa;  
};
```

name	roll	cgpa
2010	2110	2114

Structures are stored in contiguous memory location.

★ Advantages of Structures :-

- Save us from creating too many variables.
- Good data management and organization.

Array of structures :-

```
struct student ECE [100];  
struct student COE [100];  
struct student IT [100];
```

ACCESS

```
IT[0].roll = 200;  
IT[0].cgpa = 7.6;
```

For eg :-

```
#include <stdio.h>  
#include <string.h>
```

```
struct student {  
    int roll;  
    float cgpa;  
    char name[100];  
};
```

```
int main () {  
    struct student ec [100];  
    ec[0].roll = 31;  
    ec[0].cgpa = 9.5;  
    strcpy(ec[0].name, "Ankurh");  
    printf("name = %s\n", ec[0].name);  
    return 0;  
}
```

★ Initializing structures

```
struct student s1 = { "Ankurh", 31, 9.5 };  
struct student s2 = { "Rajat", 52, 7.8 };  
struct student s3 = { 0 };
```

For eg :-

```
#include <stdio.h>  
#include <string.h>
```

```
struct student {  
    int roll;  
    float cgpa;  
    char name[100];  
};
```

```
int main () {  
    struct student s1 = { 31, 9.5, "Ankurh" };  
    printf("Student roll = %d\n", s1.roll);  
    return 0;  
}
```

★ Pointers to structure

```
struct student s1;  
struct student *ptr;  
ptr = &s1;
```

```
int a;  
int *ptr;  
ptr = &a;
```


for eg :-

Above is same as previous

```
int main () {  
    struct student *ptr = &s1;  
    printf ("Student Roll = \"%d\\n\"", (*ptr).roll);  
    return 0;  
}
```

★ Arrow Operator :-

(*ptr).code $\longleftrightarrow$ ptr $\rightarrow$ code

Passing structure to functions

// Function Prototype

```
void printInfo (struct student s1);
```

for eg :-

```
#include <stdio.h>  
#include <string.h>
```

```
struct student {  
    int roll;  
    float cgpa;  
    char name;  
};
```

```
void printInfo (struct student s1);
```

```
int main () {  
    struct student s1 = {1231, 9.5, "Ankur"};  
    printInfo (s1);  
    return 0;  
}
```

```
void printInfo (struct student s1) {  
    printf ("Student Info : \\n");  
    printf ("Student roll = \"%d\\n\"", s1.roll);  
    printf ("Student cgpa = \"%f\\n\"", s1.cgpa);  
    printf ("Student name = \"%s\\n\"", s1.name);  
}
```

★ typedef keyword :-

↓
used to create alias for data types.

```
typedef struct Computer Engineering student {  
    int roll;  
    float cgpa;  
    char name[100];  
} coe;
```

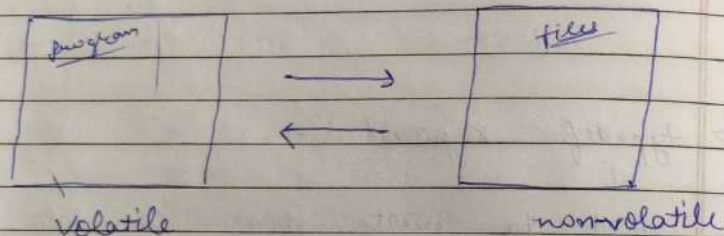
```
coe student 1;
```


Q) Enter address (house no, block, city, state) of 5 people.

★ File IO (File Input output)

RAM

Hard Disk



★ File - container in a storage device to store data.

- RAM is volatile.
- Contents are lost when program terminates.
- Files are used to persist the data.

★ Operation on Files

Create a File

Open a File

Close a File

Read from a File

Write in a File

★ Types of File

Text files

textual data

.txt, .c

Binary files

binary data

.exe, .mp3, .jpg

★ File pointer :-

File is a (hidden) structure that needs to be created for opening a file.

A file ptr that points to this structure & is used to access the file.

FILE *fptr;

★ Opening a FILE

FILE *fptr;

fptr = fopen("filename", "mode");

closing a FILE

fclose(fptr);

For eg :-

```
#include <stdio.h>
```

```
int main () {
```

```
FILE *fptr;
```

```
fptr = fopen("Test.txt", "a");
```

```
fclose(fptr);
```

```
return 0;
```

```
}
```

★ File opening Modes

"r" :- open to read

"rb" :- open to read in binary

"w" :- open to write

"wb" :- open to write in binary

"a" :- open to append

→ NULL

creat

BEST Practice :-

check if a file exist before reading from it.

For eg :- #include <stdio.h>

```
int main () {
```

```
FILE *fptr;
```

```
fptr = fopen("NewTest.txt", "r");
```

```
if (fptr == NULL) {
```

```
printf("file doesn't exist");
```

```
}
```

```
fclose(fptr);
```

```
}
```

```
return 0;
```

```
}
```

★ Reading from a file

```
char ch;
```

```
fscanf(fptr, "%c", &ch);
```

```
#include <stdio.h>
```

```
int main () {
```

```
FILE *fptr;
```

```
fptr = fopen(fptr,
```

For eg :-

```
#include <stdio.h>
```

```
int main() {
```

```
FILE *fptr;
```

```
fptr = fopen("Test.txt", "r");
```

```
int ch;
```

```
fscanf(fptr, "%c", &ch);
```

```
printf("character = %c", ch);
```

```
fclose(fptr);
```

```
return 0;
```

```
}
```


★ writing to a file

char ch = 'A';

fputc(fptr, "%c", ch);

Read & write a char

fgetc(fptr)

char read

fputc('A', fptr)

char write

★ EOF (End of File)

fgetc returns EOF to show that the file has ended.

Q) Make a program to read 5 integers from a file.

```
#include <stdio.h>
```

```
int main()
```

```
FILE *ptr;
```

```
ptr = fopen("test.txt", "r");
```

```
int n;
```

```
fscanf(ptr, "%d", &n);
```

```
printf("number = %d\n", n);
```

```
fscanf(ptr, "%d", &n);  
printf("number = %d\n", n);  
same in another three  
fclose(ptr);  
return 0;  
}
```

Q) write a program to input student information from a user & enter it to a file.

```
#include <stdio.h>
```

```
int main()
```

```
FILE *ptr;
```

```
ptr = fopen("Student.txt", "w");
```

```
char name[100];
```

```
printf("Enter name: ");
```

```
scanf("%s", name);
```

```
printf("Enter age: ");
```

```
int age;
```

```
float marks;
```

```
printf("Enter marks: ");
```

```
scanf("%s", name);
```

```
printf("Enter age: ");
```

```
scanf("%d", &age);
```

```
printf("Enter cpa: ");
```

```
scanf("%f", &cpa);
```

```
fprintf(ptr, "name is %s\n", name);  
fprintf(ptr, "age is %d", age);
```


Object oriented Programming

Object oriented programming is a programming approach that organises code into objects which combine data and functions.

Class :- A class is a undefined or a blueprint that defines the properties and behaviours of an object.

Object :- An object is an instance of a class. when you create an object, memory is allocated for its data member and you can also use it to call the class's function.

- Class → Blueprint (like a car design)
- Object → Real Thing (like an actual car)

multiplication

$$\begin{array}{r} 11 \\ \times 11 \\ \hline 11 \\ 11 \\ \hline 1001 \end{array}$$

$$\begin{array}{r} 1111 \\ \times 1111 \\ \hline 1111 \\ 1111 \\ 1111 \\ 1111 \\ \hline 11010001 \end{array}$$

Multiplication by computer method
 $(1101)_2 \times (110)_2$

$$\begin{array}{r} 01100000 \\ \downarrow \\ \text{1st shift} \quad 01100000 \\ + 0 \\ \hline 11000000 \\ \downarrow \\ \text{2nd shift} \quad 11000000 \\ + 1101 \\ \hline 10001101 \end{array}$$

$$\begin{array}{r} \text{3rd shift} \quad 00011010 \\ + 1101 \\ \hline 00100111 \end{array}$$

$$\begin{array}{r} \text{4th shift} \quad 01001110 \\ + 0 \\ \hline 101001110_2 \end{array}$$

Q. $(1101)_2$ to $(1011)_2$ by computer method

$$\begin{array}{r} 01011000 \\ \downarrow \\ \text{1st shift} \quad 01011000 \\ + 0 \\ \hline 10110000 \\ \text{2nd shift} \quad 10110000 \\ + 1101 \\ \hline 01101101 \\ \text{3rd shift} \quad 01101101 \\ + 0 \\ \hline 11011010 \\ \text{4th shift} \quad 11011010 \\ + 1101 \\ \hline 11000001_2 \end{array}$$

$$(11000001)_2$$

1 0 1 1 0 0 0 0

1st shift 1 0 1 1 0 0 0 0
+ 1 1 0 1
0 1 1 0 1 1 0 1

2nd shift 0 1 1 0 1 1 0 1
+ 0
1 1 0 1 1 0 1 0

3rd shift 1 1 0 1 1 0 1 0
+ 1 1 0 1
1 1 0 0 0 0 0 1

4th shift 1 1 0 0 0 0 1 0
+ 1 1 0 1
(1 0 0 0 1 1 1 1)
2

correct

★ auto in C
Default storage class

- Every local variable inside a function is auto by default.
- You don't need to write auto keyword, even if you write, nothing changes.

Scope

It is destroyed when the function finishes

Only inside the function where it is declared.

★ static

- keeps its value b/w function calls.
- A static variable is created only once.
- It retains its previous value until the program ends.

Scope

Same as auto, but lifetime is entire program.

★ Assembler :- An assembler is a system software that converts assembly language code into machine code. It is used in compilation process of C program.

★ Compiler :- compiler converts C code into assembly language.

★ Interpreter :- An interpreter is a program that executes program code line by line, translating each statement into machine code at runtime.

★ compiler :- A compiler converts a whole program written in a high level language into machine language. so, the computer can run it.

Q) Difference b/w compiler and interpreter.

Compiler	Interpreter
i) Translate the whole program at once.	i) Translates line by line.
ii) Execution is faster.	ii) Execution is slower.
iii) Errors are shown after entire compilation.	iii) Errors are shown immediately line by line.
iv) Used by languages like C, C++, Java.	iv) Used by languages like Python, PHP, JavaScript.
v) Needs to compile again after changes.	v) No need to recompile.

Q) What are operating system. Name any operating system?

Ans: An operating system is a system software that manages computer hardware and software resources and provide services to programs.

Q) Mobile operating system :- Mobile operating system are used in smart phones and tablets. They are designed for touch screens and small devices. Examples

Android and IOS.

ii) Network operating system :- Network operating system manages a group of computers connected in a network. It helps users share files, printers and resources safely. It is usually used in offices and schools where many computers are connected.

iii) Multiprogramming operating system :- In multiprogramming operating system, many programs stays in memory and the CPU works on one program while others wait. When one program is waiting for input, the CPU immediately works on another. This helps the computer use its time efficiently and not sit idle.

★ Identifier :- Identifiers are names you give to things in a C program.

For eg:-

int age;	age → identifier
float salary;	salary → identifier
void display();	display → identifier

Q) What is an expression in C?

Ans - An expression in C is a combination of values, variables and operators that produces a result.

Examples -

- i) Arithmetic expression
- ii) Assignment expression
- iii) Logical expression
- iv) Increment/Decrement expression

Q) What are references in C++?

Ans - A reference in C++ is another name given to an existing variable.

Ex:-

int x = 10;	Output
int &ref = x;	ref = 20;

Q) Why storage classes are necessary?

- Ans:-
- i) To control the lifetime of a variable.
 - ii) To control the visibility (scope).
 - iii) To improve performance.
 - iv) To ~~manage~~ manage memory properly.

Q) Difference b/w struct and class?

Struct	Class
i) members are public by default.	i) members are private by default.
ii) It does not support inheritance.	ii) It supports inheritance.
iii) value type, stored on the stack.	iii) Reference type, stored on the heap.
iv) The keyword "struct" defines a structure.	iv) The keyword "class" defines a class.
v) Syntax:	v) Syntax:

Q) Difference b/w struct and union.

Struct	Union
i) Structure starts with struct keyword.	i) Union starts with 'union' keyword.
ii) All the elements of the structure use different memory location.	

Q Difference b/w Struct and union

Structure	Union
i) structure start with 'struct' keyword.	i) union start with 'union' keyword.
ii) All the element of the structure use different memory location.	ii) All the elements use the single memory location for storing the data.
iii) It is commonly used in most of the applications.	iii) It is not commonly used.
iv) Syntax :- <pre>struct <struct name> { <data type> <variable> ; }</pre>	iv) Syntax :- <pre>union <union name> { <data type> <variable> ; }</pre>

Q Difference b/w structure and array

Structure	Array
i) Structure is a group of similar or not similar element data types.	i) Array is a group of elements of similar datatype.
ii) We can use the dot operator to access the elements of the struct.	ii) We can use the index of array to access the element.
iii) Structure start with 'struct' keyword.	iii) It does not start with any keyword.
iv) Eg :- <pre>struct student { char name[20]; int roll; };</pre>	iv) Eg :- <pre>int a[10];</pre>

C++ is a object oriented programming.

★ class :- class is a collection of data and functions. class is a logical method to organize data and the function in a same structure.

★ object :- object is an instance of a class which is instantiated to access the member of class.

★ Ex of class :-

```
class class name { Private:
    // data members
    Public:
    // member function
};
```

★ Basic Input/output in C++ :-

C	C++
printf() ←	cout << "Enter your Name";
scanf() ←	cin >> student name;

Q) Explain polymorphism with ~~an~~ Example.
OR

Explain function overloading with Example.
Function overloading

Sol: ~~Polymorphism~~ is a logical method of calling several function with different arguments and data types. That performs basically identical things by the same name.

eg :- #include <iostream>
using namespace std;
void Add (int , int);
void Add (int , int , int);
void Add (int , int , int , int);
void main ()
{

int a=9, b=5, c=6, d=7;

Add (a, b);

Add (a, b, c);

Add (a, b, c, d);

}

void Add (int a, int b)

{

cout << "Sum of a and b = " << a+b;

}

void Add (int a, int b, int c)

{

cout << "Sum of a, b, c = " << a+b+c;

}


```

void Add (int a, int b, int c, int d)
{
    cout << "Sum of a, b, c, d = " << a+b+c+d;
}

```

★ Encapsulation // data hiding :-

Encapsulation is the process of wrapping the same code in one place.

OR

It is the concept of hiding the data to avoid the accidental access to the data.

It is done using keyword private, public and protected.

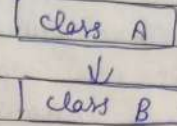
Q Explain inheritance and its type.

Ans:- Inheritance is the process of creating new class from existing class. The existing class is known as base class and newly created class is known as derived class.

- ① Reusability of the code
- ② to increase reusability of code

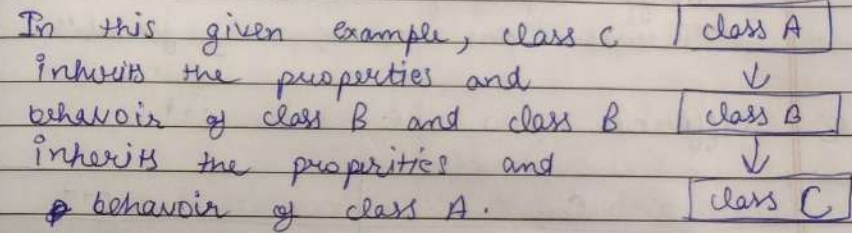
Types of Inheritance

- ① Single Inheritance :- In this inheritance, a derived class is created from a single base class.



② Multilevel Inheritance :-

In this inheritance, a derived class is created from another derived class.



★ Structure of C++ program

```

#include <iostream> // header file
using namespace std; // Standard namespace
int main () { // main function starts
    // variable declaration
    // statements
    cout << "Hello world";
    return 0;
}

```

- Q Why do we need header file in C & C++.
- Soln For i) ~~Declaration~~ Declaration of function and variable
- ii) Code reusability.

① Friend function :- The function which is used to access the private member of the class is known as friend function.

② Polymorphism :- Polymorphism in C++ is a feature that allows one function, operator or object to behave in multiple ways depending on the context. It means "one name, many forms".

③ Difference b/w C and C++.

C	C++
i) C is procedure oriented programming.	i) C++ is not object oriented programming.
ii) It focuses on function.	ii) It focuses on object and classes.
iii) Follows a top-down approach.	iii) Follows a bottom-up approach.
iv) Data is less secure.	iv) Data is more secure.
v) We use printf() and scanf() for input/output.	v) We use cout and cin for input/output.

vij) we can use <stdio.h> header file in C.

vii) Polymorphism and inheritance is not supported in C language.

viii) It does not support function and operator overloading.

ix) eg :-
#include <stdio.h>
int main() {
printf("Hello world");
return 0;
}

④ Difference b/w High level and low level language?

High level	Low level
i) This language is understood by human beings but not by machines.	i) This language is understood by machines but not by humans.
ii) High level language is like an English language.	ii) Low level language is in binary form like 0's and 1's.

vij) we use <iostream> header file in C++.

vii) polymorphism and inheritance is supported in C++.

viii) It supports function and operator overloading.

ix) eg :-
#include <iostream>
using namespace std;
int main() {
cout << "Hello world";
return 0;
}

iii) It is easy for us.

iii) It is easy for computer.

iv) It is machine independent.

iv) It is machine dependent.

v) It is easy to debug.

v) It is difficult to debug.

Q) write a program to swap two no. by call by value and call by reference.

sol: Call by value :-

```
#include <stdio.h>
```

```
void swap (int x, int y);
```

```
int main ( ) {
```

```
int a, b;
```

```
printf("Enter value of a and b");
```

```
scanf ("%d %d", &a, &b);
```

```
printf("Before swapping : a=%d, b=%d\n", a, b);
```

```
swap (a, b);
```

```
printf("After swapping : a=%d, b=%d\n", a, b);
```

```
return 0;
```

```
}
```

```
void swap (int x, int y) {  
int temp;
```

```
temp = x;
```

```
x = y;
```

```
y = temp;
```

```
printf("Inside swap function: x=%d, y=%d\n",  
x, y);
```

```
}
```

Call by reference :-

```
#include <stdio.h>
```

```
void swap (int* x, int* y);
```

```
int main ( ) {
```

```
int a, b;
```

```
printf("Enter value of a and b");
```

```
scanf ("%d %d", &a, &b);
```

```
printf("Before swapping : a=%d, b=%d\n", a, b);
```

```
swap (&a, &b);
```

```
printf("After swapping : a=%d, b=%d\n", a, b);
```

```
return 0;
```

```
}
```

```
void swap (int* x, int* y) {  
int temp;
```



```
temp = *x;
*x = *y;
*y = temp;
y
```

Q) write a program in C to product of two arrays.

```
#include <stdio.h>
int main () {
    int A[3][3] = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };
    int B[3][3] = { {2, 9, 8}, {7, 4, 6}, {5, 4, 3}, {2, 2, 4} };
    int C[3][3] = {0};
    int i, j, k;
```

```
for (i = 0; i < 3; i++)
    for (j = 0; j < 3; j++) {
        C[i][j] = 0;
        for (k = 0; k < 3; k++)
            C[i][j] += A[i][k] * B[k][j];
    }
```

```
for (i = 0; i < 3; i++) {
    for (j = 0; j < 3; j++)
        printf ("%d", C[i][j]);
    printf ("\n");
}
```

y

Q) write a program to find transpose of a matrix.

```
#include <stdio.h>
int main () {
    int A[3][3], T[3][3];
    int r, c;
```

```
printf ("Enter the rows and columns");
scanf ("%d %d", &r, &c);
```

```
printf ("Enter elements of matrix");
```

```
for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) {
```

```
scanf ("%d", &A[i][j]);
```

y

```
for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) {
```

```
T[j][i] = A[i][j];
```

y

```
printf ("Transpose of matrix\n");
for (int i = 0; i < c; i++) {
    for (int j = 0; j < r; j++) {
```

```
scanf ("%d", &T[j][i]);
```



```
printf("\n");
return 0;
}
```

Conversion

Binary (0-1), Octal (0-7), Decimal (0-9), Hexadecimal (0-F)

1) Binary to any base

a) Binary to octal

$$(11011110)_2 = (33.6)_8$$

$$(110100)_2 = (6.44)_8$$

b) Binary to decimal

$$(10110)_2 = (22)_{10}$$

$$0 \times 1 + 2 \times 1 +$$

$$1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

$$16 + 0 + 4 + 2 = 22$$

$$(110101)_2 = (53)_{10}$$

$$1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$32 + 16 + 4 + 1 = 53$$

$$(1101.11)_2 = (13.75)_{10}$$

$$1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2}$$

$$8 + 4 + 1 + \frac{1}{2} + \frac{1}{4}$$

$$13 + \frac{4+2}{8}$$

$$13 + \frac{6}{8}$$

$$13 + \frac{3}{4} = \frac{52+3}{4} = \frac{55}{4} = 13.75$$

c) Binary to Hexadecimal

$$(11011)_2 = (B)_{16}$$

842

$$0 = 0000$$

$$1 = 0001$$

$$2 = 0010$$

$$3 = 0011$$

$$4 = 0100$$

$$5 = 0101$$

$$6 = 0110$$

$$7 = 0111$$

$$8 = 1000$$

$$9 = 1001$$

$$A = 1010$$

$$B = 1011$$

$$C = 1100$$

$$D = 1101$$

$$E = 1110$$

$$F = 1111$$

0
1
2
3
4
5
6
7
8
9
A
B
C
D
E
F

$$(1101.1101)_2 = (D.D)_{16}$$

Convert Octal to Binary

$$(33.6)_8 = (011011.110)_2$$

Octal to hexadecimal

$$(237.61)_8 = (9F.C4)_{16}$$

Firstly convert into binary

$$(01001111.110100) = (9F.C4)_{16}$$

Octal to decimal

$$(564)_8 = (372)_{10}$$

$$5 \times 8^2 + 6 \times 8^1 + 4 \times 8^0$$

$$320 + 48 + 4$$

$$(462.12)_8 = (306.15)_{10}$$

$$4 \times 8^2 + 6 \times 8^1 + 2 \times 8^0 + 1 \times 8^{-1} + 2 \times 8^{-2}$$

$$256 + 48 + 2 + \frac{1}{2} + \frac{2}{4}$$

$$306 + \frac{1}{2} + \frac{1}{2}$$

$$306 + \frac{32+8}{256}$$

$$306 + \frac{40}{256}$$

$$\frac{256}{118} 59$$

$$306 + \frac{10}{59}$$

$$306.15$$

$$\frac{18054+10}{59} = \frac{18064}{59}$$

Convert decimal to Binary

$$(19.35)_{10} = (10011.0100)_2$$

$$2 \mid 19 - 18 = 1$$

$$2 \mid 9 - 8 = 1$$

$$2 \mid 4 - 4 = 0$$

$$2 \mid 2 - 2 = 0$$

$$1 = 1$$

$$.35 \times 2$$

$$= .70$$

$$= .70 \times 2$$

$$= 1.40$$

$$.40 \times 2 = 0$$

$$.80 \times 2$$

$$= 1.60$$

$$= 1$$

$$(84.12)_{10} = (1010100.0001)_{10}$$

$$2 \mid 84 - 84 = 0$$

$$2 \mid 42 - 42 = 0$$

$$2 \mid 21 - 20 = 1$$

$$2 \mid 10 - 10 = 0$$

$$2 \mid 5 - 4 = 1$$

$$2 \mid 2 - 2 = 0$$

$$1 = 1$$

$$.12 \times 2$$

$$= .24$$

$$= .24 \times 2$$

$$= .48$$

$$= .48 \times 2$$

$$= .96$$

$$= .96 \times 2$$

$$1.92$$

$$= 1$$

★ ~~Convert~~ decimal to octal

$$(19.35)_{10} = (23.26)_8$$

$\div 8$ $\times 8$

★ Convert decimal to hexadecimal

$$(19.35)_{10} = (13.59)_{16}$$

$\div 16$ $\times 16$

★ Convert hexadecimal to Binary

$$(9F.C4)_{16} = (10011111.1100100)_{2}$$

★ Convert hexadecimal to octal

$$(9F.C4)_{16} = (237.61)_8$$

$$(9F.C4)_{16} = (10011111.1100100)_{2} = (237.61)_8$$

★ Convert hexadecimal to decimal

$$(A7.C)_{16} = (167.75)_{10}$$

$$A \times 16^1 + 7 \times 16^0 + C \times 16^{-1}$$

$$160 + 7 + \frac{123}{16} = 160 + 7 + \frac{3}{4} = 167.75$$

★ Addition

$$0+0=0$$

$$0+1=1$$

$$1+0=1$$

$$1+1=10$$

$$\begin{array}{r} 1011 \\ 0110 \\ \hline 1001 \end{array}$$

$$\begin{array}{r} 1011 \\ 0110 \\ \hline 10001 \end{array}$$

$$\begin{array}{r} 1110 \\ 0110 \\ \hline 10100 \end{array}$$

$$\begin{array}{r} 1010 \\ 0011 \\ \hline 1101 \end{array}$$

$$\begin{array}{r} 1110 \\ 1100 \\ \hline 11010 \end{array}$$

$$\begin{array}{r} 110 \cdot 110 \\ 1100 \cdot 011 \\ \hline 11011 \cdot 001 \end{array}$$

$$\begin{array}{r} 111100 \\ 010101 \\ \hline 1010001 \end{array}$$

★

Subtraction

$$0 - 0 = 0$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

$$0 - 1 = 1$$

$$\begin{array}{r} 1110 \\ 0101 \\ \hline 1010 \end{array}$$

$$\begin{array}{r} 1100 \\ 0101 \\ \hline 1001 \end{array}$$

$$\begin{array}{r} 1100 \\ 1001 \\ \hline 0011 \end{array}$$

$$\begin{array}{r} 010 \\ 1110 \\ 0101 \\ \hline 1001 \end{array}$$

$$\begin{array}{r} 101 \\ 1100 \\ 0101 \\ \hline 0111 \end{array}$$

$$\begin{array}{r} 0 \\ 11000 \\ 00001 \\ \hline 10111 \end{array}$$

$$\begin{array}{r} 1011 \\ 11000 \\ 01111 \\ \hline 01001 \end{array}$$

★ 1's and 2's Complement

e) 101011

1's complement = 010100

2's complement = 010100
1
010101 Ans

For direct find 2's complement

$$\begin{array}{r} 101011 \\ \leftarrow \\ \hline 010101 \text{ Ans} \end{array}$$

Q) 10100

Find 2's complement

2's 01100

1's 01011

Multiplication

$$\begin{array}{r}
 0110 \\
 \times 0101 \\
 \hline
 0110 \\
 0000x \\
 0110x \\
 1x \\
 \hline
 0000x \\
 0000x \\
 001110
 \end{array}$$

* Multiplication by computer method

$$(1101)_2 \times (110)_2$$

$$\begin{array}{r}
 01101000 \\
 \downarrow \\
 \text{1st shift} \quad 01100000 \\
 \hline
 11000000
 \end{array}$$

$$\begin{array}{r}
 \text{2nd shift} \quad 10100000 \\
 \hline
 1101 \\
 10001101
 \end{array}$$

$$\begin{array}{r}
 \text{3rd shift} \quad 100001010 \\
 \hline
 1101 \\
 00100111
 \end{array}$$

$$\begin{array}{r}
 \text{4th shift} \quad 001001110 \\
 \hline
 0 \\
 01001110
 \end{array}$$

$$\text{Ans} = (01001110)_2$$

$$\text{ex) } (110)_2 \text{ to } (1011)_2$$

$$11011000$$

$$\begin{array}{r}
 \text{1st shift} \quad 10110000 \\
 \hline
 101 \\
 01101101
 \end{array}$$

$$\begin{array}{r}
 \text{2nd shift} \quad 011011010 \\
 \hline
 0 \\
 11011010
 \end{array}$$

$$\begin{array}{r}
 \text{3rd shift} \quad 110110100 \\
 \hline
 1101 \\
 11000001
 \end{array}$$

$$\begin{array}{r}
 \text{4th shift} \quad 110000110 \\
 \hline
 1101 \\
 10000111
 \end{array}$$

Ans

Q) $(0110)_2$ to $(0101)_2$

$[0 | 1 | 0 | 1 | 0 | 0 | 0 | 0]$

1st shift $0 [1 | 0 | 1 | 0 | 0 | 0 | 0 | 0]$

$1 | 0 | 1 | 0 | 0 | 0 | 0 | 0$

2nd shift $1 [0 | 1 | 0 | 0 | 0 | 0 | 0 | 0]$

$0 | 1 | 0 | 0 | 0 | 0 | 1 | 0$

3rd shift $0 [1 | 0 | 0 | 0 | 0 | 1 | 1 | 0]$

$1 | 0 | 0 | 0 | 1 | 1 | 0 | 0$

4th shift $1 [0 | 0 | 0 | 1 | 1 | 0 | 0 | 0]$

$0 | 1 | 1 | 0$

$(0 | 0 | 0 | 1 | 1 | 1 | 0 | 0)$

2 An

★ High level language :- A high level language is a programming language that is easy to understand, uses English like statement and details. C is called a high level language because it is ~~sa~~ human readable and easy to write compared to low level language or assembly language.

★ Low level language :- A low level language is a programming language that is close to machine language and directly interacts with computer hardware. It is difficult for humans to read and write.

★ Assembly language :- Assembly language is a low-level programming language that uses short symbolic words (called mnemonics) like MOV, ADD, SUB instead of binary code. It is easier than machine language but still closely related to hardware.

★ operating system :-

Batch System

Multiprogramming

Time sharing system

Real time system

Network operating system

Distributed operating system

Ans :- An operating system (OS) is the main software that controls the entire computer and helps users to run application. There are different types of operating systems :-

i) Batch operating System :- A batch operating system collects many jobs and

runs after them one after another without the user controlling it directly. Users give their work to the computer, and the computer stores all the jobs and processes them in groups called ~~batches~~ batches.

ii) Multiprogramming operating system :- A multiprogramming operating system keeps many programs in memory at the same time. When one program is waiting, the CPU immediately starts working on another program. This way the computer is always busy and does not waste time.

iii) Time sharing :- A time sharing operating system allows many users to work on the same computer at a same time. Each user gets a very small time slice of the CPU, so it looks like everyone is using the system simultaneously.

iv) Real-Time :- A Real time operating system gives quick and immediate response. It is used in ~~quite~~ critical systems where delays cannot be tolerated. Examples :- Robots, medical equipment, traffic control system.

v) Network operating system :- A network operating system ~~na~~ manages computers connected within a network. It helps in sharing

files, applications and communication b/w different devices. It is commonly used in business network.

vi) Distributed operating system :- A distributed operating system connects many computers together so they work as one single system. Tasks are shared among multiple machines, which increases speed and efficiency.

C++

* First program :-

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World";
    return 0;
}
```

* Basic Structure of C++ :-

```
#include <iostream> * header file
using namespace std;
int main() {
    cout << "Hello World";
    return 0;
}
```

* Variables :- Containers to store your data.
int, float, char

* Comment :-
// (Single line comment)
/* → */ (Multi line comment)

* Variable types in C++:

1. int
2. float
3. char
4. double
5. boolean

* Based on scope, variable can be classified into two types:

1. Local variable
2. Global variable

* Scope of a variable is the region in code where the existence of variable is valid.

* Local variable :- Local variable are declared inside the braces of any function and can be accessed only from there.

* Global variable :- Global variable are declared outside any function and can be accessed from anywhere.

* Can global and local variable have same name in C++? : Yes, we will see.